You can use Prometheus in two ways (that is, you're using the same binary, with different configurations):

- Prometheus server (default mode)
- Prometheus agent mode (https://prometheus.io/blog/2021/11/16/agent/), optimized for scraping and remote read–write for federation

We saw Prometheus in action already in chapter 2, but as a reminder, the core of Prometheus is its scrape config. This is where you define what Prometheus should scrape (its targets) and how it is supposed to find the targets (service discovery) as well as, optionally, recording and alerting rules. For example, to scrape the metrics that Prometheus itself exposes (i.e., self-scraping), you could use something like the following in your scrape config:

```
global:
  scrape_interval: 30s

scrape_configs:
  - job_name: "prom"
    static_configs:
      - targets: ["localhost:9090"]
```

By default, Prometheus scrapes (calls) targets every 15 seconds, but here, we relax it to 30.

Prometheus adds the job name as a label (here, job=prom) to any metrics time series scraped. This is a provenance good practice and can aid correlation.

We're using a static service discovery method (providing the URL to scrape or call). In this case, since Prometheus exposes the metrics under $host:9090/metrics, we use this as a value for the targets key.

Note that scrape configs continue to be important, going forward, since OpenTelemetry aims to be compatible with Prometheus and allows you to reuse your scrape configs (with minor adaptations). You will see all of this in action in the end-to-end example in this chapter.

If you want to learn more about how to use Prometheus as well as how it works internally, I strongly recommend the ultimate reference: *Prometheus: Up & Running*, by Julien Pivotto and Brian Brazil, *Second Edition* (O'Reilly; http://mng.bz/x458).

4.2.2 Other metrics agents

While Prometheus is the established metrics platform in the cloud-native ecosystem, especially in the Kubernetes context, there are other agents you may come across, for various reasons, including legacy usage or because your organization decided to go all in with a certain vendor or project. This includes, but is not limited to, the following agents (in alphabetical order):

- *Collectd (https://collectd.org/)*—Written in C, with this agent, you can collect (operating) system and application performance metrics efficiently. Several cloud providers and ISVs provide integrations with collectd, such as Amazon CloudWatch, Splunk, and Sumo Logic.
- *Grafana Agent (https://grafana.com/docs/agent/latest/)*—Created by Grafana Labs, this agent is optimized for Prometheus metrics. The code for this agent was donated to the Prometheus project and forms the basis of the agent mode in